

Open in app ↗



Search

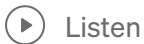


How to tackle dataset class imbalance for NLP



Ding Han · Follow

7 min read · Jul 1, 2023



Listen



Share



More

Photo by [Christophe Hautier](#) on [Unsplash](#)

In the field of machine learning, class imbalance poses a significant challenge that can significantly impact the performance of predictive models. When the distribution of classes in a dataset is highly skewed, standard learning algorithms tend to be biased towards the majority class, leading to poor predictions for the minority class. While techniques like Synthetic Minority Over-sampling Technique (SMOTE) have been widely used to address class imbalance, this article aims to shed

light on the importance of weighted loss functions in comparison with oversampling and undersampling approaches.

What is Class Imbalance

Class imbalance occurs when the number of instances in one class significantly outweighs the number of instances in the other class(es). This can lead to biased models that prioritize the majority class and neglect the minority class. In real-world scenarios, such as fraud detection or rare disease diagnosis, the minority class is often of paramount interest due to its prevalence. Addressing class imbalance is crucial to achieve accurate and reliable predictions.

Here is a look at the dataset we are going to use for this example:

IMDB Dataset of 50K Movie Reviews

Large Movie Review Dataset

www.kaggle.com

Premise

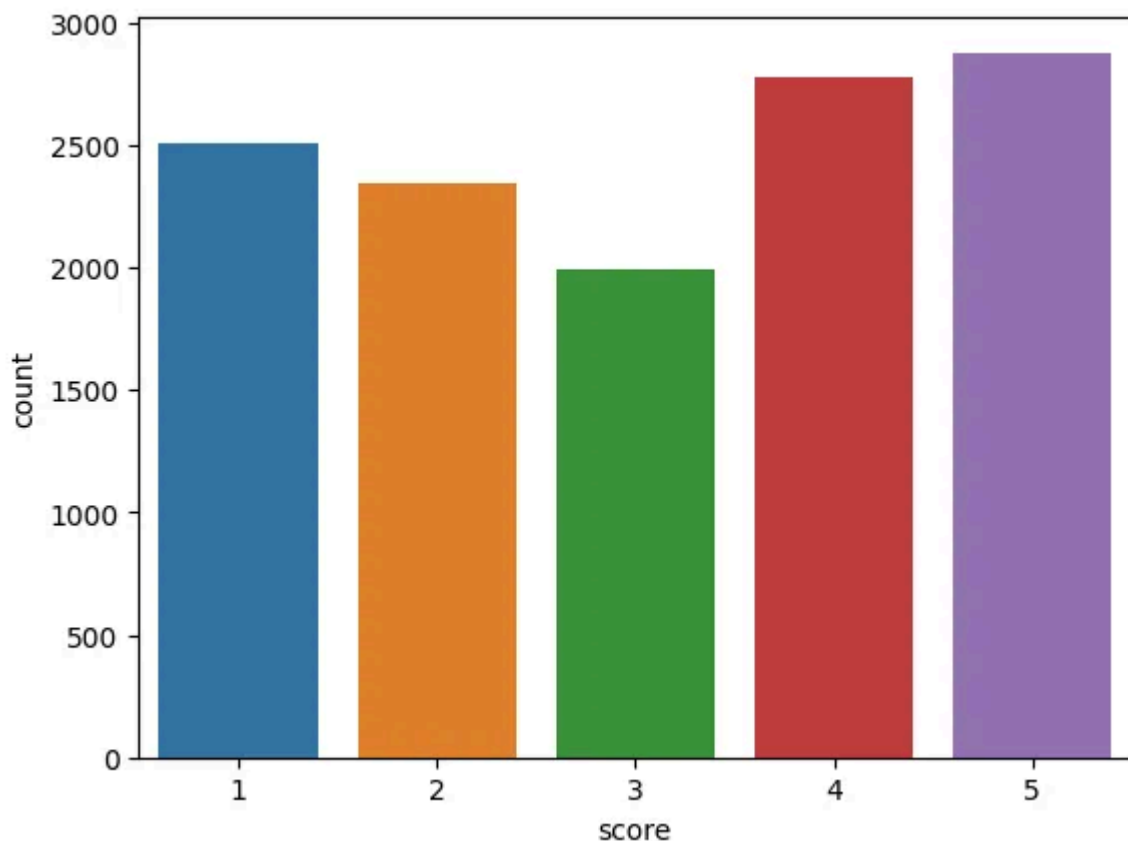
Let's say we are given a multi-class classification task for classifying the sentiment of the user based on the score (label) and content.

```
import pandas as pd
df = pd.read_csv("reviews.csv")
df[['content', 'score']]
```

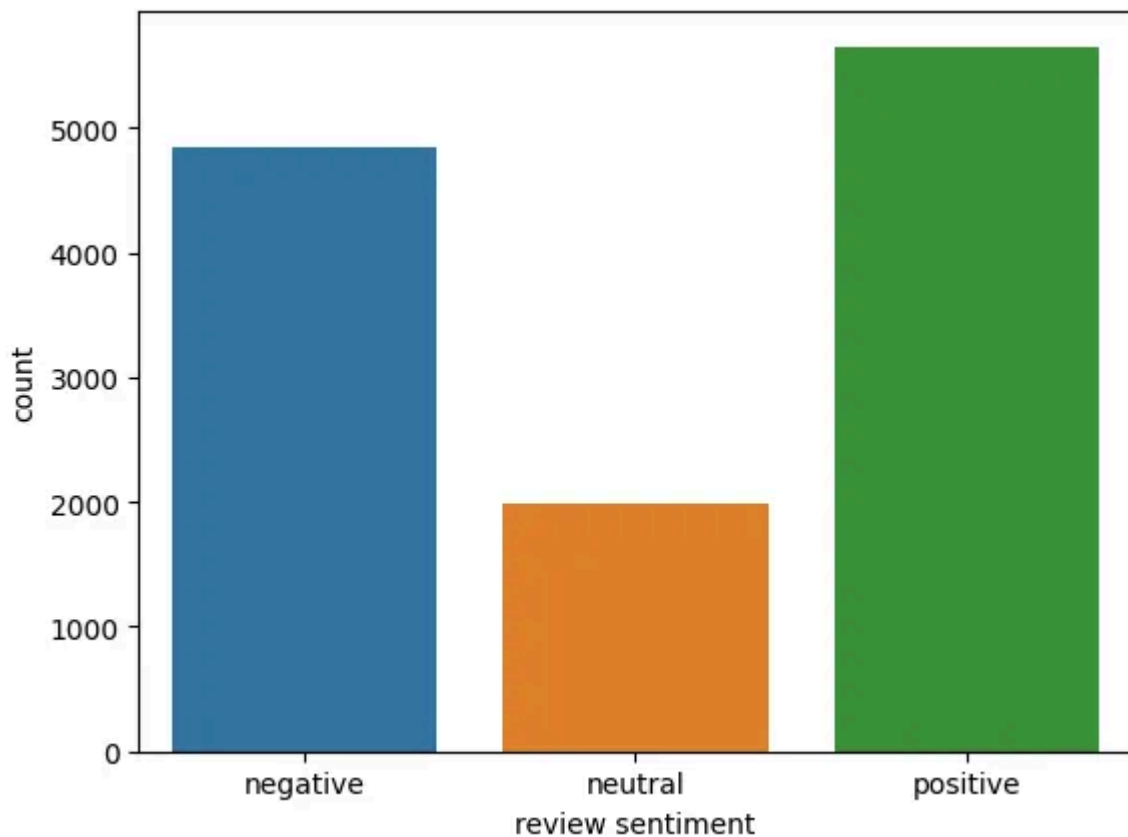
	content	score
0	I cannot open the app anymore	1
1	I have been begging for a refund from this app...	1
2	Very costly for the premium version (approx In...	1
3	Used to keep me organized, but all the 2020 UP...	1
4	Dan Birthday Oct 28	1
...	...	...
12490	I really like the planner, it helps me achieve...	5
12491	😊****😊	5
12492	Very useful apps. You must try it	5
12493	Would pay for this if there were even more add...	5
12494	Sooow good	5

12495 rows × 2 columns

Since there are 5 different labels as shown below, we will be compressing them to 3 different classes (negative, neutral, and positive).



```
def to_sentiment(rating):  
    rating = int(rating)  
    if rating <= 2:  
        return 0  
    elif rating == 3:  
        return 1  
    else:  
        return 2  
  
df['sentiment'] = df.score.apply(to_sentiment)
```



After pre-processing the labels, we can clearly see there is a significant amount of class imbalance for the neutral sentiment. We will look into the different ways to tackle this issue later.

Using the above sentiment label, we will train a multi-class classifier with the content. Below is our baseline score after training it with the bert-base-cased:

	precision	recall	f1-score	support
negative	0.78	0.77	0.78	242

neutral	0.40	0.39	0.40	125
positive	0.81	0.83	0.82	258
accuracy			0.72	625
macro avg	0.66	0.66	0.66	625
weighted avg	0.72	0.72	0.72	625

Oversampling or Undersampling

Oversampling and undersampling techniques are commonly employed to address class imbalance. Oversampling involves replicating instances from the minority class to increase their representation, while undersampling reduces the instances from the majority class to achieve a balanced dataset. By combining oversampling and undersampling, we can enhance the representation of the minority class while reducing the dominance of the majority class.

Undersampling for the majority classes or Oversampling for the minority classes is one of the most commonly used approaches for managing Class Imbalance in a dataset.

When we implement Majority Class Undersampling, we essentially eliminate a specific amount of samples from the Majority classes. Oversampling for minority classes, on the other hand, involves repeating samples from minority groups.

Although these solutions balance the dataset, they do not directly address the problems produced by Class Imbalance and instead risk adding new problems. Oversampling introduces redundant samples, which can quickly slow down training and lead to model overfitting. On the other hand, undersampling eliminates a given amount of samples. As a result, the model may miss out on learning essential ideas that could have been learned from the deleted samples due to undersampling.

```
from imblearn.under_sampling import RandomUnderSampler

# Randomly under sample the majority class
rus = RandomUnderSampler(random_state=42)
df_train_rus, _ = rus.fit_resample(df_train, df_train['sentiment'])
```

Results after undersampling were very underwhelming:

	precision	recall	f1-score	support
negative	0.00	0.00	0.00	242
neutral	0.00	0.00	0.00	125
positive	0.41	1.00	0.58	258
accuracy			0.41	625
macro avg	0.14	0.33	0.19	625
weighted avg	0.17	0.41	0.24	625

As you can see, after undersampling the majority class the results became way worst than before.

SMOTE

Synthetic Minority Oversampling Technique (SMOTE). SMOTE works by creating synthetic instances of the minority class to balance the dataset. It does this by identifying minority class instances and generating synthetic examples by interpolating between neighbouring instances.

SMOTE also operates in the feature space, which means that the generated synthetic data does not accurately represent the characteristics of the text within its feature space. NLP problems typically involve huge feature spaces, making it difficult for SMOTE, which relies on K-nearest neighbours (KNN), to effectively handle such high-dimensional data. All in all, the application of SMOTE in our sentimental analysis problem poses difficulties due to its inability to create representative synthetic data in the feature space and the limitations of KNN in handling large feature spaces.

Weighted Loss Function

Now let's look into weighted loss function. Based on the class these samples belong to, we might effectively apply various weights to the estimated loss. The objective is to weigh the loss computed for different samples differently depending on whether they are from the majority or minority groups. We essentially want to give the loss experienced by the samples linked with minor classes a higher weight.

For our case, we are dealing with a **multi-class classification** task. Let's take a look at the `CrossEntropyLoss` function in the official [Pytorch website](https://pytorch.org/docs/stable/nn.html#torch.nn.CrossEntropyLoss):

$$\ell_c(x, y) = L_c = \{l_{1,c}, \dots, l_{N,c}\}^\top$$

Where N is the batch size

$$l_n = -w_{y_n} \log \frac{\exp(x_{n,y_n})}{\sum_{c=1}^C \exp(x_{n,c})} \cdot 1\{y_n \neq \text{ignore_index}\}$$

c is the class size, n is the number of samples in the batch, w_{y_n} is the manual rescaling weight given to the loss of nth element of the batch and P_c is the weight of the positive for class c. X is the input, Y is the target.

w_{y_n} will be the sample weight that we wish to compute for every batch iteration. It must be a Tensor of size N*C and can be assigned using the 'weight' option (C is the total Number of Classes). The weights will be positive numbers, which are calculated using the proportion of samples classified with a specific class. It must be a vector with a length equal to the number of classes and weighs the contribution of each class to the Loss. "If a dataset contains 100 positive and 300 negative samples of a single class, then pos weight for the class should be equal to $300/100=3$," according to the Pytorch Documentation. The Loss will make the dataset appear to have $3 \times 100 = 300$ good examples." There are many different weighting methods out there.

In this example, we will be sklearn scikit_class_weight library, which actually uses the formula $n_samples / (n_classes * np.bincount(y))$ to derive the positive class weights.

```
def compute_class_weight(train_y):
    """
    Compute class weight given imbalanced training data
    Usually used in the neural network model to augment the loss function (weight)
    Favouring/giving more weights to the rare classes.
    """
    import sklearn.utils.class_weight as scikit_class_weight

    class_list = list(set(train_y))
    class_weight_value = scikit_class_weight.compute_class_weight(class_weight=
class_weight = dict()

    # Initialize all classes in the dictionary with weight 1
    curr_max = int(np.max(class_list))
    for i in range(curr_max):
        class_weight[i] = 1

    # Build the dictionary using the weight obtained the scikit function
```

```
for i in range(len(class_list)):
    class_weight[class_list[i]] = class_weight_value[i]

return class_weight
```

```
compute_class_weight(df_train['sentiment'])
```

✓ 0.1s

```
{0: 0.8587247040855288, 1: 2.124905517762661, 2: 0.7326687516288767}
```

After which, we will convert them from list to tensor and parse them into our loss function:

```
import torch

weight_list = []
for key, weight in weights.items():
    weight_list.append(weight)
weight_tensor = torch.FloatTensor(weight_list)

# with weights
loss_fn = nn.CrossEntropyLoss(weight=weight_tensor).to(device)
```

Below is the result after performing weighted loss function:

	precision	recall	f1-score	support
negative	0.77	0.82	0.80	242
neutral	0.52	0.40	0.45	125
positive	0.81	0.85	0.83	258
accuracy			0.75	625
macro avg	0.70	0.69	0.69	625
weighted avg	0.74	0.75	0.74	625

As we can see the macro avg for the F1 has **increased** by 4.5% after weighting the loss based on the sample size differences

Side Note (multi-label classification)

For **multi-label classification** method, the loss function used was PyTorch implementation of Binary Cross Entropy which combines a sigmoid layer (`torch.nn.BCEWithLogitLoss`). The function can be mathematically expressed as shown in the official [Pytorch website](https://pytorch.org/docs/stable/nn.html#torch.nn.BCEWithLogitLoss):

$$\ell_c(x, y) = L_c = \{l_{1,c}, \dots, l_{N,c}\}^\top$$

Where N is the batch size

$$l_{n,c} = -w_{n,c} [p_c y_{n,c} \cdot \log \sigma(x_{n,c}) + (1 - y_{n,c}) \cdot \log(1 - \sigma(x_{n,c}))]$$

c is the class size, n is the number of samples in the batch, $w_{n,c}$ is the manual rescaling weight given to the loss of nth element of the batch and p_c is the weight of the positive for class c

For our project, we will be using the inverse of the number of samples (INS). This method inverse the class frequency to which the class belongs.

Conclusion

In conclusion, the choice between using a weighted loss function versus oversampling and undersampling techniques depends on the specific needs and constraints of the problem at hand. Weighted loss functions can be effective when dealing with imbalanced datasets, as they assign higher importance to minority classes, thereby providing a direct optimization approach. On the other hand, oversampling and undersampling techniques can be useful for rebalancing the dataset, but they may introduce bias or result in information loss.

With the rise of generative AI and Language Models (LLMs), we now have the ability to generate synthetic datasets, even for classes with lower sample sizes. By leveraging the power of LLMs, we can generate synthetic data points that closely resemble the characteristics and patterns of the minority class. This synthetic data can then be combined with the original dataset, increasing the representation of the minority class and improving the model's performance.

Ultimately, the choice between using a weighted loss function, oversampling/undersampling, or generating synthetic data depends on factors such as the dataset size, class imbalance severity, available computational resources, and the specific problem domain. It is important to carefully evaluate and experiment

with different approaches to determine the most suitable method for addressing class imbalance and achieving optimal model performance.

Code: <https://github.com/odh1995/medium-weighted-loss>

Pytorch

AI

Dataset

Class Imbalance

Classification



Follow



Written by Ding Han

19 Followers

AI Engineer @ MSD
